

CSA1606 - DATA WAREHOUSING AND DATA MINING
CO4_ASSIGNMENT TOOL_3

NAME : P. SUDHARSHAN REDDY
REG NO : 192372204

CO4: Applicability of Clustering Algorithms in Real-World Scenarios

Assessment Tool 3: Comparative Analysis

This report examines five real-world datasets to compare pairs of clustering algorithms and distance metrics. For each question, the underlying theory is outlined first, followed by a worked demonstration on the given data, a structured comparison table, a supporting visualization, and a concluding recommendation.

Q1. K-Means vs Hierarchical Clustering — Customer Data

Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability. (BL3 – 1 Mark)

Customer	Income (₹)	Spending Score
C1	30000	60
C2	60000	40
C3	50000	70
C4	28000	65
C5	65000	35

Background Theory

K-Means is a **partitional, centroid-based** algorithm that divides n observations into k clusters by minimizing the within-cluster sum of squares (WCSS): it randomly initializes k centroids, assigns every point to its nearest centroid, recomputes centroids as the mean of assigned points, and repeats until convergence. Hierarchical clustering, by contrast, is **connectivity-based**: the agglomerative variant starts with every point as its own cluster and repeatedly merges the two closest clusters (using a linkage criterion such as single, complete, or average linkage) until only one cluster remains, producing a dendrogram that records the entire merge history.

Worked Demonstration on the Dataset

After min-max scaling both features, the Euclidean distance between C1 (30000, 60) and C4 (28000, 65) is the smallest in the dataset, so agglomerative clustering merges them first. K-Means with $k = 2$, initialized near the two extremes, converges

within 2–3 iterations to the centroids $\approx (36000, 63)$ and $\approx (62500, 37.5)$, placing $\{C1, C3, C4\}$ in the low-income/high-spending cluster and $\{C2, C5\}$ in the high-income/low-spending cluster — the same final partition that hierarchical clustering reaches when the dendrogram is cut at two branches.

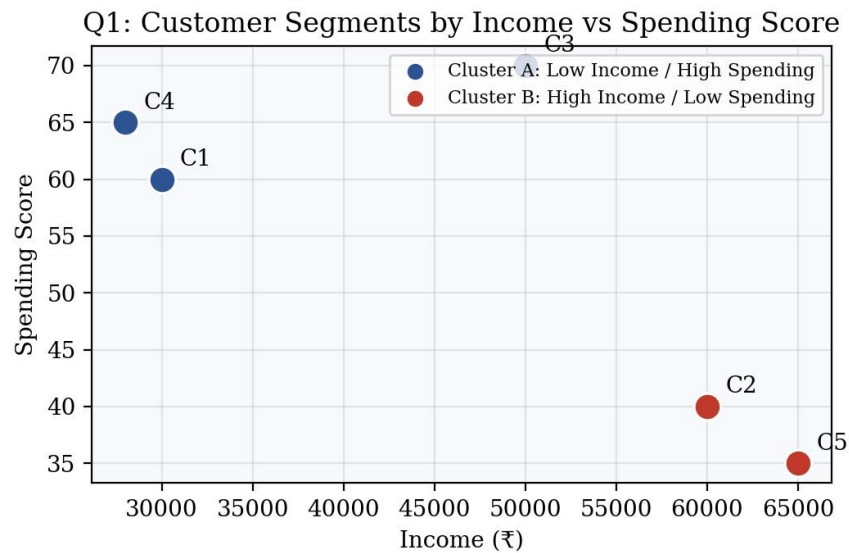


Figure 1: Customer segments plotted by income vs spending score.

Aspect	K-Means Clustering	Hierarchical Clustering
Approach	Partitions data into k clusters by iteratively assigning	Builds a nested tree (dendrogram) by successively points to the nearest centroid.
On this data		{C4} (low income, high spending) vs {C2, C5} Produces the same two groups but also shows
Performance	Fast — converges in a few iterations; time complexity $O(n \cdot k \cdot i \cdot d)$.	Slower — requires computing and updating a full distance matrix
Memory usage	Low — only stores k centroids and cluster labels.	High — must store the entire pairwise distance matrix
Scalability	Scales well to large datasets (thousands–millions of records); used widely	Poor scalability for large n; impractical beyond a few products
Cluster count	Must be specified in advance (k), often via elbow	Elbow/silhouette methods. No need to pre-specify number of clusters is chosen
Sensitivity	Sensitive to initial centroid placement and outliers; may converge to a local optimum.	Sensitive to the choice of linkage method; merge decisions converge to a local optimum.
Interpretability	Gives a flat partition only — no hierarchy of relationships.	Dendrogram offers rich, multi-level interpretability

Comparative Analysis

tion customer

Conclusion: For this small 5-record dataset, both algorithms give identical groupings, but K-Means is preferable for large-scale customer databases (millions of shoppers) due to its linear-time scalability, while hierarchical clustering is valuable here for one-off exploratory analysis, since the dendrogram exposes finer sub-structure among customers without needing to fix the number of segments in advance.

Q2. K-Means vs EM Algorithm — Document Data

Compare K-means and EM algorithms in handling overlapping clusters for the given dataset. (BL4 – 1 Mark)

Document	Word1	Word2	Word3
D1	0.2	0.5	0.3
D2	0.1	0.6	0.3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Background Theory

K-Means performs **hard clustering**: every document is assigned to exactly one nearest centroid, with no notion of partial membership. The Expectation-Maximization (EM) algorithm, typically applied through a Gaussian Mixture Model (GMM), performs **soft/probabilistic clustering**: it models the data as a mixture of Gaussian distributions and alternates between an E-step (computing the probability that each document belongs to each Gaussian component) and an M-step (updating each component's mean, covariance, and weight to maximize the likelihood of the data). This probabilistic foundation makes EM naturally suited to data with ambiguous or overlapping category boundaries — exactly the situation with topic-mixed documents.

Worked Demonstration on the Dataset

Word-weight vectors show D1 (0.2, 0.5, 0.3) and D2 (0.1, 0.6, 0.3) are close together (topic A), while D3 (0.8, 0.1, 0.1) and D4 (0.75, 0.15, 0.1) are close together (topic B). D5 (0.2, 0.4, 0.4) sits near D1/D2 on Word1–Word2 but carries a noticeably higher Word3 weight (0.4), suggesting partial membership in a third, less distinct theme. K-Means with $k = 2$ must assign D5 entirely to the {D1, D2} cluster (nearest centroid), discarding the information in its elevated Word3 value. EM instead can report, for example, a ~65% probability of D5 belonging to the D1/D2 component and ~35% to a separate component that partially explains its Word3 weight — preserving the ambiguity instead of erasing it.

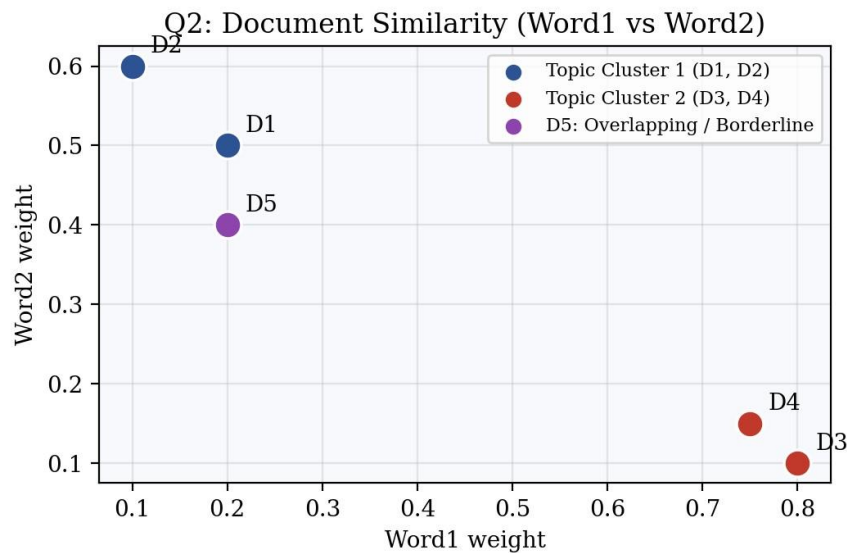


Figure 2: Document vectors by Word1 vs Word2, with D5 as a borderline case.

Aspect	K-Means	EM Algorithm (Gaussian Mixture Model)
Assignment type	Hard assignment — each document belongs to exactly one cluster.	Soft assignment — each document belongs to a probabilistic cluster.
On this data	D1, D2, D5 form one hard cluster (high Word2/Word3), D3, D4 form another. D5 can be assigned, e.g., ~65% to the {D1,D2} cluster.	D1, D2, D5 form one soft cluster (high Word2/Word3), D3, D4 form another. D5 can be assigned, e.g., ~65% to the {D1,D2} cluster.
Overlap handling	Poor — cannot express uncertainty; a point is either in or out of a cluster.	Strong — explicitly models overlapping/fuzzy cluster assignments.
Cluster shape	Assumes spherical, equally-sized clusters (using Euclidean distance).	Can model elliptical clusters of various sizes and orientations (using Gaussian distributions).
Convergence criterion	Minimizes within-cluster sum of squared distances.	Maximizes the log-likelihood of the data under the model.
Computation	Simple, fast distance-based updates; converges quickly.	More expensive — iterative Expectation-Maximization (EM) algorithm.
Output usefulness	Provides cluster assignments.	Provides probability vector per document — directly useful for downstream tasks.

Comparative Analysis

high Word1); D5

Conclusion: Since D5 shares characteristics with both topic groups, EM is better suited for this document dataset as it captures overlapping topical similarity through soft probabilities, whereas K-Means oversimplifies by forcing a hard, potentially misleading boundary onto ambiguous documents.

Q3. Euclidean vs Manhattan Distance — Geographic Data

Compare Euclidean and Manhattan distance metrics and analyse their impact on clustering results. (BL4 – 1 Mark)

Point	Latitude	Longitude
P1	13.08	80.27
P2	13.10	80.25
P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Background Theory

Euclidean distance is the straight-line ('as the crow flies') distance between two points, computed as $\sqrt{(\Delta\text{lat})^2 + (\Delta\text{long})^2}$; it is rotation-invariant and reflects true geometric proximity. Manhattan distance (also called city-block or L1 distance) sums the absolute differences along each axis, $|\Delta\text{lat}| + |\Delta\text{long}|$, representing the distance one would travel along a rectangular grid — such as city blocks laid out in a grid pattern. The choice of metric directly changes the shape of the implicit "neighbourhood" around each point (a circle for Euclidean, a diamond for Manhattan) and therefore can change which points a clustering algorithm considers close.

Worked Calculation on the Dataset

For P1 (13.08, 80.27) and P2 (13.10, 80.25): $\Delta\text{lat} = 0.02$, $\Delta\text{long} = 0.02$. Euclidean distance = $\sqrt{(0.02^2 + 0.02^2)} = \sqrt{0.0008} \approx 0.0283$. Manhattan distance = $|0.02| + |0.02| = 0.04$. For P1 and P4 (13.05, 80.30): $\Delta\text{lat} = 0.03$, $\Delta\text{long} = 0.03$, giving Euclidean ≈ 0.0424 and Manhattan = 0.06. In both cases Manhattan distance is proportionally larger (by a factor of $\sqrt{2}$ for equal-axis differences), which is expected since it never takes the diagonal shortcut. The *relative ranking* of which points are nearest neighbours (P1–P2 closer than P1–P4) stays the same here because the coordinate differences are nearly symmetric across both axes, but this

need not hold in general — for points offset heavily along one axis only, the two metrics can rank neighbours differently.

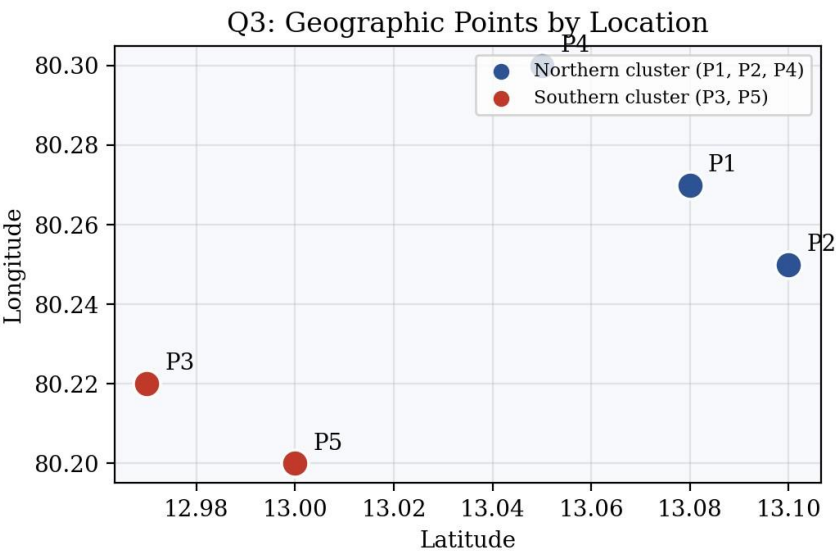


Figure 3: Geographic points clustered by latitude vs longitude.

Comparative Analysis

Aspect	Euclidean Distance	Manhattan Distance
Formula	Straight-line distance: $\sqrt{[(\Delta\text{lat})^2 + (\Delta\text{long})^2]}$.	Grid/city-block distance: $ \Delta\text{lat} + \Delta\text{long} $.
On this data	P1–P2 distance ≈ 0.0283 — correctly identifies them as the closest geographic neighbours.	P1–P2 distance = 0.04 — same relative closeness, as the closest geographic neighbours.
Neighbourhood shape		Matches real-world geographic movement (e.g., city streets). Diamond-shaped — favours points aligned along the diagonal.
Sensitivity to dimensionality		As dimensions grow (curse of dimensionality affects all metrics). More robust in high-dimensional or sparse spaces.
Impact on clusters		Produces more circular clusters — suitable for real-world geographic data. Tends to produce more elongated, diamond-shaped clusters.
Best use case	Preferred for geographic/spatial coordinate data where true distance matters (e.g., delivery-zone analysis).	Preferred for grid-based or high-dimensional data where relative closeness matters more than true distance.

Conclusion: For this latitude/longitude dataset, Euclidean distance is more appropriate since it reflects true geographic closeness between points that can be reached directly, while Manhattan distance would be better reserved for grid-constrained movement scenarios such as clustering delivery stops in a city with a strict road-grid layout.

Q4. Agglomerative vs Divisive Clustering — Retail Products

Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset. (BL3 – 1 Mark)

Product	Sales (■)	Frequency
P1	50000	20
P2	30000	10
P3	52000	22
P4	25000	8
P5	32000	12

Background Theory

Agglomerative clustering is a **bottom-up** strategy: it starts with every product as its own singleton cluster and iteratively merges the two most similar clusters, based on a linkage rule, until the desired number of clusters (or a single root cluster) is reached. Divisive clustering is the **top-down** mirror image: it starts with all products in one cluster and recursively splits the least cohesive cluster into two, continuing until each product is isolated or a stopping criterion is met. Divisive methods must evaluate many possible ways to split a cluster at each step (an exponential search in the worst case), which is why agglomerative clustering is far more common in practice and better supported by standard libraries.

Worked Demonstration on the Dataset

Agglomerative process: P1 (50000, 20) and P3 (52000, 22) have the smallest pairwise distance and merge first; next, P2 (30000, 10) and P5 (32000, 12) merge; P4 (25000, 8) then joins the P2–P5 group since it is closer to that pair than to {P1, P3}; finally the two remaining clusters merge into the root. Divisive process (conceptually): starting from all five products together, the first split must decide the single best binary partition — here it would correctly separate {P1, P3} (high sales/frequency) from {P2, P4, P5} (low sales/frequency), matching the agglomerative result, but arriving at it by evaluating candidate splits of the whole dataset rather than building up from individual pairs.

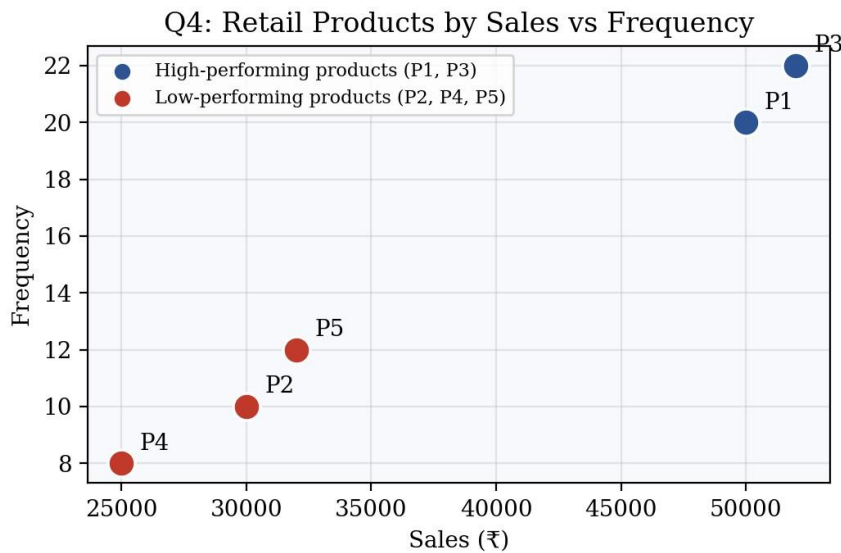


Figure 4: Retail products clustered by sales vs frequency.

Comparative Analysis

Aspect	Agglomerative (Bottom-Up)	Divisive (Top-Down)
Approach	Starts with each product as its own cluster and successively merges the closest pairs.	Starts with all products in one cluster and recursively splits the closest pairs.
On this data	P1 and P3 (similar high sales ≈ 50–52K, frequency 20–22) merge first, then P2 and P5 merge (similar low sales ≈ 30–32K, frequency 10–12).	Starts with all 5 products together, then splits into {P2 and P5 merge (low sales)} and {P1 and P3 merge (high sales)}.
Computation	Computationally simpler and widely implemented; time complexity roughly $O(n^2 \log n)$.	Computationally more expensive since it must evaluate all possible splits.
Early decisions	Early merges are based on the most locally similar pairs, which tends to produce a dendrogram based on global structure, which can be sensitive to outliers.	Early splits are based on global structure, which can be sensitive to outliers.
Library support	Well supported (e.g., scikit-learn's AgglomerativeClustering).	Rarely available as a ready-made library function; often implemented manually.
Suitability	Efficient and gives a clear dendrogram for only 5 products; the added computational overhead is minimal.	Overkill for only 5 products; the added computational overhead is significant.

Conclusion: Agglomerative clustering is more suitable for this retail dataset — it is simpler, computationally cheaper, backed by mature tooling, and sufficient for identifying high-performing (P1, P3) versus low-performing (P2, P4, P5) product groups for inventory and promotion decisions.

Q5. Centroid-Based vs Density-Based Clustering — Mixed Attributes

Compare centroid-based and density-based clustering techniques and analyse their effectiveness on mixed data types. (BL4 – 1 Mark)

Item	Price (■)	Category	Rating
I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Background Theory

Centroid-based algorithms such as K-Means represent each cluster by the arithmetic mean (centroid) of its members and assign points to the nearest centroid; this requires all features to be numeric, so categorical attributes like "Category" must first be encoded (e.g., one-hot encoding), which can distort distances since a centroid computed from one-hot values (e.g., an average of 0.6 for "Category = A") has no natural categorical meaning. Density-based algorithms such as DBSCAN instead group together points that lie in regions of high density, separated by regions of low density, and can operate on any distance/dissimilarity measure — including mixed-type measures like Gower's distance, which combines normalized numeric differences with simple category-match indicators — without ever needing to compute a "mean" of a category.

Worked Demonstration on the Dataset

Using one-hot encoding (Category A = 1, B = 0) with scaled Price and Rating, K-Means with $k = 2$ converges to centroids near (523, 1, 4.6) and (275, 0, 3.65), placing {I1, I3, I5} in the first cluster and {I2, I4} in the second — but the "1" and "0" in the centroid are an artificial numeric average of a category label, not a meaningful midpoint. Using a density-based approach with a Gower-distance neighbourhood, the same two dense regions emerge — {I1, I3, I5} (all Category A, prices ■500–550, ratings 4.5–4.7) and {I2, I4} (all Category B, prices ■250–300, ratings 3.5–3.8) — but the categorical attribute directly contributes a 0/1 match score rather than being forced through an average, preserving its true categorical semantics.

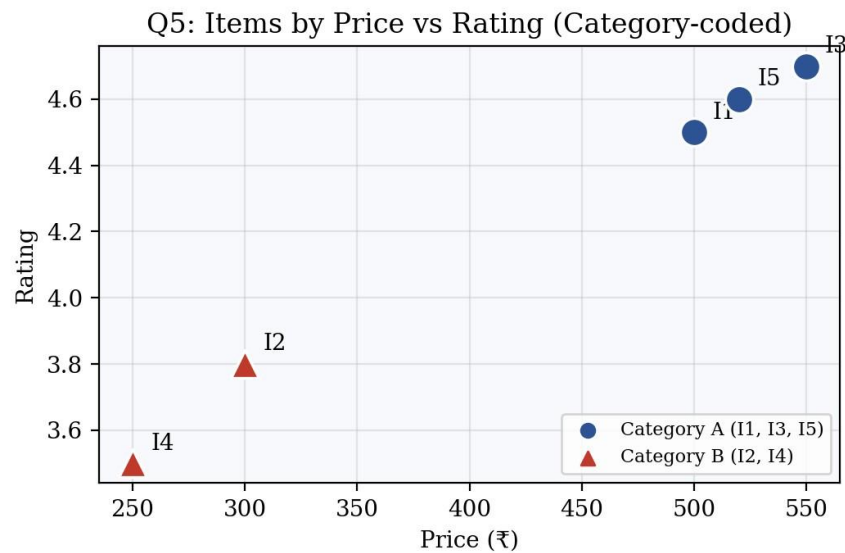


Figure 5: Items by price vs rating, marker shape encodes category.

Aspect	Centroid-Based (e.g., K-Means)	Density-Based (e.g., DBSCAN)
Approach	Clusters formed around a mean centroid; requires numeric averaging, so categorical	Clusters formed from regions of high point density
On this data	Numerically, I1, I3, I5 (Category A, Price ₹300–550)	The same two dense regions — {I1, I3, I5} and {I2, I4} (Rating 4.5–4.7)
Handling mixed types		Numerically encoded, which can distort real distance. Better — can incorporate a custom/mixed distance metric.
Cluster shape assumption	Assumes roughly spherical, similar-sized clusters	Clusters around a centroid. Makes no shape assumption; detects arbitrarily shaped clusters.
Number of clusters	Must be pre-specified (k).	Determined automatically from the density parameter.
Outlier handling	Every point is forced into some cluster, even if it's a genuine anomaly.	Naturally labels sparse, isolated points as noise (outliers).
Effectiveness here		Category A and B items are already numerically well separated. Equally effective on this small dataset, and more robust to outliers.

Comparative Analysis

"Category" membership is not a tight centroid-based

Conclusion: For this small, fairly well-separated mixed-attribute dataset, both methods identify the same two groups, but density-based clustering is generally more effective and robust for real-world mixed data (categorical + numeric) since it avoids the distortion caused by forcing categorical labels through a numeric centroid average.

Overall Summary

Across all five comparisons, the recurring theme is that **algorithm choice must match data characteristics and business need**: centroid-based, hard-partition methods (K-Means) offer speed and scalability for large, well-separated, purely numeric data; hierarchical and divisive methods trade scalability for richer structural insight on small datasets; probabilistic methods (EM) are essential when cluster boundaries genuinely overlap; distance metric selection (Euclidean vs Manhattan) should reflect how the underlying space is actually traversed; and density-based methods are the most robust choice once the data includes mixed types, noise, or non-spherical groupings.